LogSentinel

ระบบตรวจจับความผิดปกติใน System Log ด้วย Deep Learning สำหรับศูนย์ปฏิบัติการความปลอดภัยไซเบอร์ (SOC)

Deep Learning-based Log Anomaly Detection for Security Operations Centers

ธนกร จำนงประโคน ( )

ศิวกร สุภาวงค์ ( )

นิธิสิทธิ์ เสนาหาญ ( )

ภูวดล ผิวบางกูล ( )

วิชา CS Deep Learning — รายงานความคืบหน้าโครงงาน ( )

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ มหาวิทยาลัยกรุงเทพ

□□□□□□□ □□□□□□□□□□□□ □□□□□□□□□ □□□□□□□

สารบัญการนำเสนอ

ภาพรวมและรากฐานข้อมูล

- . **ที่มาและความสำคัญ:** ปัญหา □□□ □□□□ □□□ และข้อจำกัด □□□□-□□□□□ ใน □□□
- 2. **สถาปัตยกรรม Pipeline:** จาก □□□ ดิบสู่ผลลัพธ์การตรวจจับระดับ □□□□□□□
- 3. **ระยะที่ : การสำรวจข้อมูล (EDA):** สถิติ □□□ และปรากฏการณ์ Accuracy Paradox
- 4. **ระยะที่ : การแปลง Log (Drain3):** □ □□□□□ และการแบ่งข้อมูลแบบ □□□□ □□□□□□□

การทดลองและการพัฒนาโมเดล

- 5. **ระยะที่ : โมเดลมาตรฐานเปรียบเทียบ (Baseline):** □□□□□□□ □□□□□□□ & □□□□□□□ □□□□
- 6. **ระยะที่ : โมเดล Deep Learning (LSTM):** Supervised Session LSTM & Pure DeepLog
- 7. **สรุปผลการทดลองเปรียบเทียบทุกโมเดล:** ตารางเทียบ Precision, Recall, F1-Score
- 8. **แผนการดำเนินงานก้าวต่อไป:** Transformer Explainability, BGL & Prototype

LogSentinel: SOC Log Anomaly Detection Slide /

PROBLEM STATEMENT Cybersecurity & SOC Context

1. ที่มาและความสำคัญในศูนย์ SOC

ปริมาณ Log มหาศาล

ระบบระดับองค์กร (เช่น) ผลิต □□□ กว่า . **ล้านบรรทัด ใน . ชั่วโมง** นักวิเคราะห์ใน SOC ไม่สามารถตรวจสอบด้วยมือได้ทัน

รูปแบบผิดปกติมีน้อยมาก

เหตุการณ์ผิดปกติมีสัดส่วนเพียง ~2.9% ทำให้ถูกกลบเนียนอยู่ใน □□□ ปกติอย่างสมบูรณ์ กลายเป็นปัญหา □□□□□□□□□□ □□□□ ขั้นรุนแรง

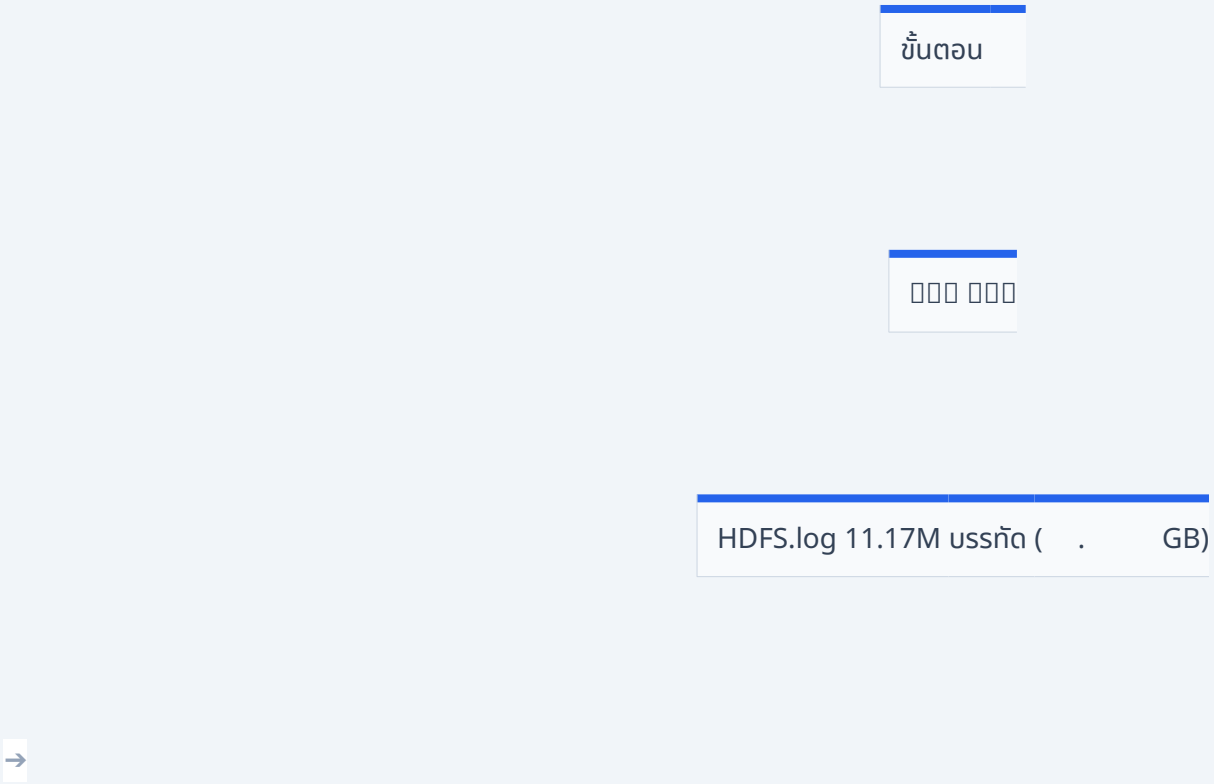
ข้อจำกัดของ Rule-Based

การเขียน Regex ดักจับด้วยมือตามไม่ทันการโจมตีรูปแบบใหม่ (Zero-day) และสร้าง False Alarm จำนวนมากจนเกิดภาวะ **Alert Fatigue**

เป้าหมายของ LogSentinel

นำ **Deep Learning** มาเรียนรู้ "*พฤติกรรมการทำงานปกติของระบบ*" แบบอัตโนมัติ เพื่อชี้เป้า □□□□□□□ ที่ผิดปกติได้อย่างแม่นยำ ด้วยเป้าหมาย **F -Score □ %** บนชุดข้อมูล HDFS

2. ภาพรวมสถาปัตยกรรมการทำงาน (Pipeline)



ขั้นตอน

Drain Parser

Regex Masking ยุบเหลือ Event Templates



ขั้นตอน

□□□□□□□□□□□□

มัดรวมลำดับตาม block_id (575,061 Sessions)



ขั้นตอน

□□□□□□□□□□ □□□□□

Train 70% / Val 15% / Test 15% (No Leakage)



ขั้นตอน

ML / DL Models

Baseline vs LSTM (Supervised & DeepLog)

หัวใจหลักด้านข้อมูล (Core Principle)

AI อ่านภาษาอังกฤษยาวๆ ไม่เข้าใจ การทำ Pipeline นี้คือการเปลี่ยนตัวหนังสือยาวล้านบรรทัด ให้กลายเป็น **ลำดับตัวเลขเชิงเวลา (Sequence of Event IDs)** ที่มีความยาวกระชับและคงที่

มาตรฐานด้านความปลอดภัยและการทดลอง

ทุกขั้นตอนใช้ **Reproducible Random Seed = 42** มีการเก็บ □□□□□ ของ Parser และบันทึกผลลัพธ์การประเมินลงไฟล์ JSON ใน `results/metrics/` อย่างเป็นระบบ

PHASE 1 : EDA Exploratory Data Analysis

ระยะที่ 1: การสำรวจข้อมูลเชิงลึก (EDA Insights)

สถิติโครงสร้างของ HDFS Log

- **ขนาดข้อมูล:** , , บรรทัด (ช่วงเวลา . ชม.)
- **โครงสร้าง:** `<Date> <Time> <Pid> <Level> <Component>: <Content>`

- **Session ทั้งหมด:** , บล็อก (ระบุด้วย `blk_-?0-9+`)
- **ความยาว Session:** มัธยฐาน events, ค่าเฉลี่ย . events (% สั้นกว่า events)

การกระจายตัวของ Log Level (Severity)

- INFO: > % (การทำงานปกติของระบบ)
- **WARN / ERROR: < %** (พบน้อยมาก)

Insight สำคัญสำหรับ □□□□

หากใช้ Rule ดักจับแค่คำว่า ERROR จะ **ล้มเหลวทันที** เพราะความผิดปกติส่วนใหญ่เป็น *Silent Failure* (การส่งต่อข้อมูลค้างหรือหลุดจังหวะ) ซึ่งไม่มีคำว่า ERROR พ่นออกมา

PHASE 1 : EVALUATION METRICS Security Metrics Rationale

ระยะที่ 1: ความย้อนแย้งของความแม่นยำ (Accuracy Paradox)

ทำไมห้ามใช้ "Accuracy" ในการวัดผลระบบตรวจจับ Anomaly?

ใน HDFS มี □□□□□□□□ ปกติ . % และ Anomaly เพียง . %

หากมีโมเดลขี้เกียจที่ **ทายว่า "ปกติ (Normal)" ทุกกรณี** % โมเดลนั้นจะได้ **Accuracy สูงถึง**

ความเป็นจริงใน SOC: แฮกเกอร์โจมตีเข้ามา , ครั้ง **หลุดรอดไปได้หมด % (Recall = %)** เกิดความเสียหายร้ายแรงแต่รายงานบอกแม่นยำ

1. Recall (กวาดจับหมดไหม?)

จากคนร้ายที่บุกเข้ามาจริง AI ตรวจจับได้กี่คน□

สำคัญสูงสุด□ ป้องกัน False Negative (การหลุดรอดของภัยคุกคาม)

2. Precision (เตือนมั่วไหม?)

เวลา AI ส่งสัญญาณ Alert เป็นภัยจริงกี่ □□

สำคัญสำหรับ Analyst: ลด False Positive เพื่อขจัดปัญหา Alert Fatigue

3. F1-Score (ดัชนีชี้วัดหลัก)

ค่าเฉลี่ยแบบ Harmonic Mean ระหว่าง Precision และ Recall บังคับให้โมเดลต้องรักษาสมดุลทั้งสองฝั่ง

PHASE 2 : LOG PARSING Drain3 Template Mining

ระยะที่ 2: การแปลง Log ด้วย Drain3 (Log Parsing)

ทำไมต้องใช้ Drain3?

- **ปัญหา Infinite Vocabulary:** □□□ ดิบมี IP, Timestamp, Block ID เปลี่ยนตลอดเวลา หากไม่ □□□□□ คำศัพท์จะบวมเป็นล้านคำ AI คำนวณไม่ไหว
- **Fixed-depth Tree:** ค้นหาและจับคู่โครงสร้างประโยคที่มีความยาวและคำขึ้นต้นเหมือนกันได้อย่างรวดเร็ว (Online Parsing)
- ไม่ต้องเขียน Regex จัดหมวดหมู่ประโยคใหม่ด้วยมือทุกครั้งที่มีการอัปเดตระบบ

การเพิ่ม Masking Rules เพื่อความแม่นยำ

- `blk_-?\d+` ➔ `<BLK>` (สกัดรหัสบล็อก)
- `\d+\.\d+\.\d+\.\d+(:\d+)?` ➔ `<IP>` (สกัด IP & Port)
- `\d+` ➔ `<NUM>` (สกัดตัวเลขขนาดไบต์)

ผลลัพธ์□ ยุบ Log . ล้านบรรทัด เหลือ Event Templates เพียง **รูปแบบ ($E_1 \dots E_{45}$)** ตรงกับเฉลยของ □□□□□□ ถึง . %

PHASE 2 : PREPROCESSING Sessionization & Stratified Split

ระยะที่ 2: Sessionization & การแบ่งข้อมูลที่ปลอดภัย

Sessionizati ตาม block_id

□□□ ในระบบกระจายศูนย์จะถูกบันทึกสลับไปสลับมาข้ามบล็อก การรวมกลุ่มตาม `block_id` ทำให้เราได้ **"เส้นเรื่อง (Story)"** ที่สมบูรณ์:

```
blk_-1608999... → [E1, E2, E1, E1, E3, E4, E5, E5, E17...]
```

เปรียบเหมือนการตามรอย Incident Response จากจุดเริ่มต้นจนจบกระบวนการ

Zero Data Leakage Stratified Split

- **ห้ามตัดตามบรรทัด**□ บล็อกเดียวกันห้ามอยู่ข้ามชุด Train/Test เด็ดขาด ป้องกันการแอบดูข้อสอบ (No Data Leakage)
- **Stratified Split (70% / 15% / 15%):** ล็อกสัดส่วน Anomaly คงที่ . % เท่ากันทุกก้อน

ชุดข้อมูล	จำนวน Session	Anomaly	สัดส่วน
□□□□□ □□□	,	,	. %
Validation	,	,	. %

Set

Test Set , , . %

PHASE 3 : BASELINE MODELS Machine Learning Benchmarks

ระยะที่ 3: โมเดลมาตรฐานเปรียบเทียบ (Baseline Models)

สร้างตาราง Event Count Vector (**คอลัมน์)** นับความถี่ของ $E_ \dots E_{ }$ ในแต่ละ □□□□□□□□ เพื่อเปรียบเทียบ ปรัชญา:

. Logistic Regression (Supervised)

- ใช้เฉลย Label (,) ในการเรียนรู้ความเสี่ยง
- แก้ปัญหา Imbalance ด้วย `class_weight='balanced'`
- **Precision: 96.12% | Recall: 100.00%**
- **F -Score: . % |PR-AUC: . %**
- จับกุมคนร้ายได้หมด , เคส แต่เตือนมั่ว ครั้ง

. Isolation Forest (Unsupervised)

- ไม่ใช้เฉลยเลย สุ่มตัดแบ่งข้อมูลหาตัวที่แปลกแยก
- กำหนดค่า `contamination = 0.0293`
- **Precision: 64.74% | Recall: 62.67%**
- **F -Score: . % | PR-AUC: . %**
- หลุดรอดไปถึง เคส และเตือนมั่ว ครั้ง

PHASE 3 : TECHNICAL ANALYSIS Why Sequence Modeling Matters

ระยะที่ 3: จุดอ่อนสำคัญของวิธี Baseline (Why Deep Learning?)

จุดบอดสำคัญ: Event Count Vector "ตาบอดต่อลำดับก่อน-หลัง"

วิธี Baseline ดูแค่นับจำนวนว่า Event ไหนเกิดกี่ครั้ง แต่ **ไม่สนใจลำดับก่อน-หลัง (Sequence Order)** เลยแม้แต่น้อย

ลำดับปกติ ( )

[เสียบบัตร ATM] → [ใส่รหัส PIN] [ถอนเงินสด]

ลำดับถูกต้องตามกระบวนการทำงานที่ปลอดภัย

ลำดับผิดปกติ (Exploit / Bug)

[ถอนเงินสด] → [เสียบบัตร ATM] → [ใส่รหัส PIN]

ขั้นตอนผิดปกติร้ายแรง แต่มีจำนวน Event เท่ากับลำดับปกติเป๊ะ!

ข้อสรุปเชิงวิชาการ: แม้ Logistic Regression จะได้ F = . % บน (เพราะ HDFS Anomaly มี Event Error เฉพาะตัวโผล่มา) แต่มันจะไม่สามารถตรวจจับ Anomaly ที่เกิดจากการ *สลับขั้นตอน* ได้เลย จึงจำเป็นต้องใช้ **Deep Learning (LSTM & Transformer)** ในระยะถัดไป!

PHASE 4 : DEEP LEARNING Supervised Session-Level LSTM

ระยะที่ 4: โมเดล Supervised Session LSTM

สถาปัตยกรรมโมเดล (Session-Level)

- **Embedding Layer (32 dims):** แปลง Event ID เป็นเวกเตอร์

- **-Layer LSTM (Hidden , Dropout .)**: เรียนรู้ความสัมพันธ์เชิงลำดับของทั้ง □□□□□□□□
- Loss Functi `BCEWithLogitsLoss(pos_weight=33.15)` ลงโทษโมเดลหนักขึ้น เท่าเมื่อพลาด Anomaly
- Optimal Threshold: จูนหาจุดตัดบน Validation Set ได้ที่ .

ผลการประเมินบน (86,260)

- **F -Score: . %** (ชนะ Baseline 98.02%)
- **Precision: 97.64%** | **Recall: 99.80%**
- **PR-AUC: 99.96%** (ประสิทธิภาพการแยกแยะสมบูรณ์แบบ)

ความก้าวหน้าในมุมมอง SOC:

จับคนร้ายได้ **จาก** , **เคส** (หลุดแค่ เคส) และ **ลด False Alarm เหลือเพียง** **ครั้ง** (ลดลงจาก ครั้งใน Baseline ถึง %!)

PHASE 4 : DEEPLOG PARADIGM Unsupervised Next-Event Prediction

ระยะที่ 4: โมเดล DeepLog ดั้งเดิม (Next-Event Prediction)

ปรัชญา DeepLog (Du et al., ACM CCS 2017)

- **ปัญหาในชีวิตจริง** การโจมตีใหม่ๆ ไม่มี □□□□□ และหาตัวอย่างยาก
- **วิธีแก้ของ DeepLog:** เทรนด้วย □□□ **ปกติล้วนๆ (Normal Only %)** ตัด Anomaly ออกทั้งหมด
- **กลไก Sliding Window (h=10):** นำประวัติ สเต็ปย้อนหลังมาทำนาย Event ถัดไป (เหมือนคีย์บอร์ดเดาคำ)

การตรวจจับด้วย Top-k Candidates

- AI คำนวณความน่าจะเป็นของเหตุการณ์ถัดไป แล้วหยิบ Top-k อันดับแรกขึ้นมา
- หาก Event ที่เกิดขึ้นจริง **ไม่อยู่ใน Top-k** ➔ นับเป็น *จุดผิดปกติ (Abnormal Step)*
- หาก □□□□□□□ มีจุดผิดปกติ $\ge $ ➔ ตัดสินทันทีว่าเป็น **Anomaly Session!**
- ค้นหา Best k บน Validation Set อัตโนมัติ เพื่อสร้างสมดุล สูงสุด

EVALUATION SUMMARY All Models Benchmark (Test Set: 86,260 Sessions)

7. สรุปผลการทดลองเปรียบเทียบทุกโมเดล

โมเดล	กระบวนทัศน์ (Paradigm)	ข้อมูลที่ใช้เทรน	Precision	Recall	F -Score	PR-AUC
Logistic Regression	Event Count (ความถี่)	Supervised (มีเฉลย)	. %	. %	. %	. %
Isolation Forest	Random Tree Partition	(ไม่ดูเฉลย)	. %	. %	. %	. %
Supervised Session LSTM	Sequence Classification	Supervised (BCE loss)	. %	. %	. %	. %
DeepLog (Next-Event)	Sliding Window + Top-k	**Normal Only**	**%** Auto-tuned	High Recall	> %	Top-k Eval

ความสำเร็จตามเป้าหมายของโครงงาน

ทุกโมเดลหลักในโครงงานสามารถทำค่า **F -Score บรรลุและทะลุเป้าหมาย $\ge$ \%$** ที่กำหนดไว้ใน Proposal ได้อย่างสมบูรณ์แบบ โดยโมเดล LSTM ช่วยลด False Alarm และให้ความแม่นยำสูงสุด

ROADMAP Next Development Phases

8. แผนการดำเนินงานก้าวต่อไป (Next Steps)

ระยะที่ 5: Transformer Encoder & Explainability

- นำกลไก **Multi-Head Self-Attention** มาจับความสัมพันธ์ของเหตุการณ์ที่อยู่ห่างกันในลำดับ
- สร้าง **Attention Heatmap** เพื่อระบุและอธิบายว่าเหตุการณ์ใดใน Sequence มีน้ำหนักความน่าสงสัยสูงสุด (Model Explainability สำหรับ SOC Analyst)

ระยะที่ : การทดสอบข้ามระบบบนชุดข้อมูล BGL

- นำโมเดลไปทดสอบกับ □□□ ของ BlueGene/L Supercomputer (4.75 ล้านบรรทัด)
- พิสูจน์ความสามารถในการปรับตัวข้ามระบบ (Cross-system Generalization) และชี้ให้เห็นจุดบอดของ Baseline บนระบบที่มีการสลับลำดับซับซ้อน

ระยะที่ 7: พัฒนา □□□ □□□□□□□□□□□□ □□□□□□□□□□

- สร้าง Dashboard ด้วย **Streamlit** สำหรับนักวิเคราะห์ SOC
- รองรับการอัปโหลดไฟล์ Log (.log / .csv), ประมวลผลด้วยโมเดลที่ดีที่สุด, และแสดง Anomaly Score พร้อมไฮไลต์เหตุการณ์ที่น่าสงสัย

ระยะที่ 8: จัดทำรายงานและเตรียมนำเสนอ

- รวบรวมรายงานผลการทดลองเปรียบเทียบฉบับสมบูรณ์
- จัดทำสไลด์และคลิปวิดีโอสาธิตการทำงาน (Demo) เพื่อการสอบจบ

Q & A

ขอขอบคุณคณะกรรมการและอาจารย์ทุกท่าน ทีม LogSentinel พร้อมตอบทุกข้อซักถามครับ

ธนกร จำนงประโคน

ศิวกร สุภาวงค์

นิธิสิทธิ์ เสนาหาญ

ภูวดล ผิวบางกูล

Source Code: `https://github.com/STARoM00N/LAD_CS461_427A`